Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное и прикладное программное обеспечение

Дисциплина: Информационные системы

Курсовая работа

«trace»

Информационная система поддержки проектной и учебной деятельности студентов

Этап 2

Выполнили:
Карнажицкий М. Р., Ерохин Е. Г.
Группа: P3311

Проверил:
Тюрин И. Н.

г. Санкт-Петербург, 2026

Содержание

1. Даталогическая модель 4
   1.1. Схема 4
2. Таблицы 5
   2.1. Пользователи и проекты 5
      2.1.1. User 5
         2.1.1.1. Поля и ограничения 5
         2.1.1.2. DDL 5
         2.1.1.3. Индексы 6
      2.1.2. Project 6
         2.1.2.1. Поля и ограничения 6
         2.1.2.2. DDL 6
         2.1.2.3. Индексы 7
      2.1.3. Team 7
         2.1.3.1. Поля и ограничения 7
         2.1.3.2. DDL 7
         2.1.3.3. Индексы 8
      2.1.4. TeamMember 8
         2.1.4.1. Поля и ограничения 8
         2.1.4.2. DDL 8
         2.1.4.3. Индексы 8
      2.1.5. TeamApplication 9
         2.1.5.1. Поля и ограничения 9
         2.1.5.2. DDL 9
         2.1.5.3. Индексы 10
   2.2. Задания и назначения 10
      2.2.1. TaskTemplate 10
         2.2.1.1. Поля и ограничения 10
         2.2.1.2. DDL 11
         2.2.1.3. Индексы 11
      2.2.2. TaskParameter 11
         2.2.2.1. Поля и ограничения 11
         2.2.2.2. DDL 12
         2.2.2.3. Индексы 12
      2.2.3. Task 12
         2.2.3.1. Поля и ограничения 13
         2.2.3.2. DDL 13
         2.2.3.3. Индексы 14

2.2.4. TeamTask 14
2.2.4.1. Поля и ограничения 14
2.2.4.2. DDL 14
2.2.4.3. Индексы 15
2.2.5. TeamTaskParameterValue 15
2.2.5.1. Поля и ограничения 15
2.2.5.2. DDL 15
2.2.5.3. Индексы 16
2.3. Сдача и проверка 16
2.3.1. SubmissionAttempt 16
2.3.1.1. Поля и ограничения 16
2.3.1.2. DDL 16
2.3.1.3. Индексы 17
2.3.2. SubmissionAttachment 17
2.3.2.1. Поля и ограничения 17
2.3.2.2. DDL 17
2.3.2.3. Индексы 18
2.3.3. Review 18
2.3.3.1. Поля и ограничения 18
2.3.3.2. DDL 19
2.3.3.3. Индексы 19
2.4. Объявления и обсуждение 19
2.4.1. Announcement 19
2.4.1.1. Поля и ограничения 20
2.4.1.2. DDL 20
2.4.1.3. Индексы 20
2.4.2. DiscussionMessage 20
2.4.2.1. Поля и ограничения 21
2.4.2.2. DDL 21
2.4.2.3. Индексы 22
3. Триггеры 23
3.1. `trg_validate_default_value_type` 23
3.1.1. Зачем 23
3.1.2. Код 23
3.2. `trg_validate_team_task_project` 24
3.2.1. Зачем 24
3.2.2. Код 24
3.3. `trg_validate_team_task_parameter` 25
3.3.1. Зачем 25

3.3.2. Код ... 25
3.4. trg_validate_parameter_value ... 26
3.4.1. Зачем ... 26
3.4.2. Код ... 26
3.5. trg_validate_submission_author ... 26
3.5.1. Зачем ... 26
3.5.2. Код ... 26
3.6. trg_mark_team_task_submitted ... 27
3.6.1. Зачем ... 27
3.6.2. Код ... 27
3.7. trg_validate_reviewer ... 28
3.7.1. Зачем ... 28
3.7.2. Код ... 28
3.8. trg_update_team_task_status ... 28
3.8.1. Зачем ... 28
3.8.2. Код ... 29
3.9. trg_validate_message_scope ... 29
3.9.1. Зачем ... 29
3.9.2. Код ... 29
4. Функции и процедуры ... 31
4.1. apply_to_team ... 31
4.1.1. Зачем ... 31
4.1.2. Код ... 31
4.2. review_team_application ... 32
4.2.1. Зачем ... 32
4.2.2. Код ... 32
4.3. submit_task ... 34
4.3.1. Зачем ... 34
4.3.2. Код ... 34
4.4. review_submission ... 35
4.4.1. Зачем ... 35
4.4.2. Код ... 35
5. Заполнение тестовыми данными ... 37

1. Даталогическая модель

1.1. Схема

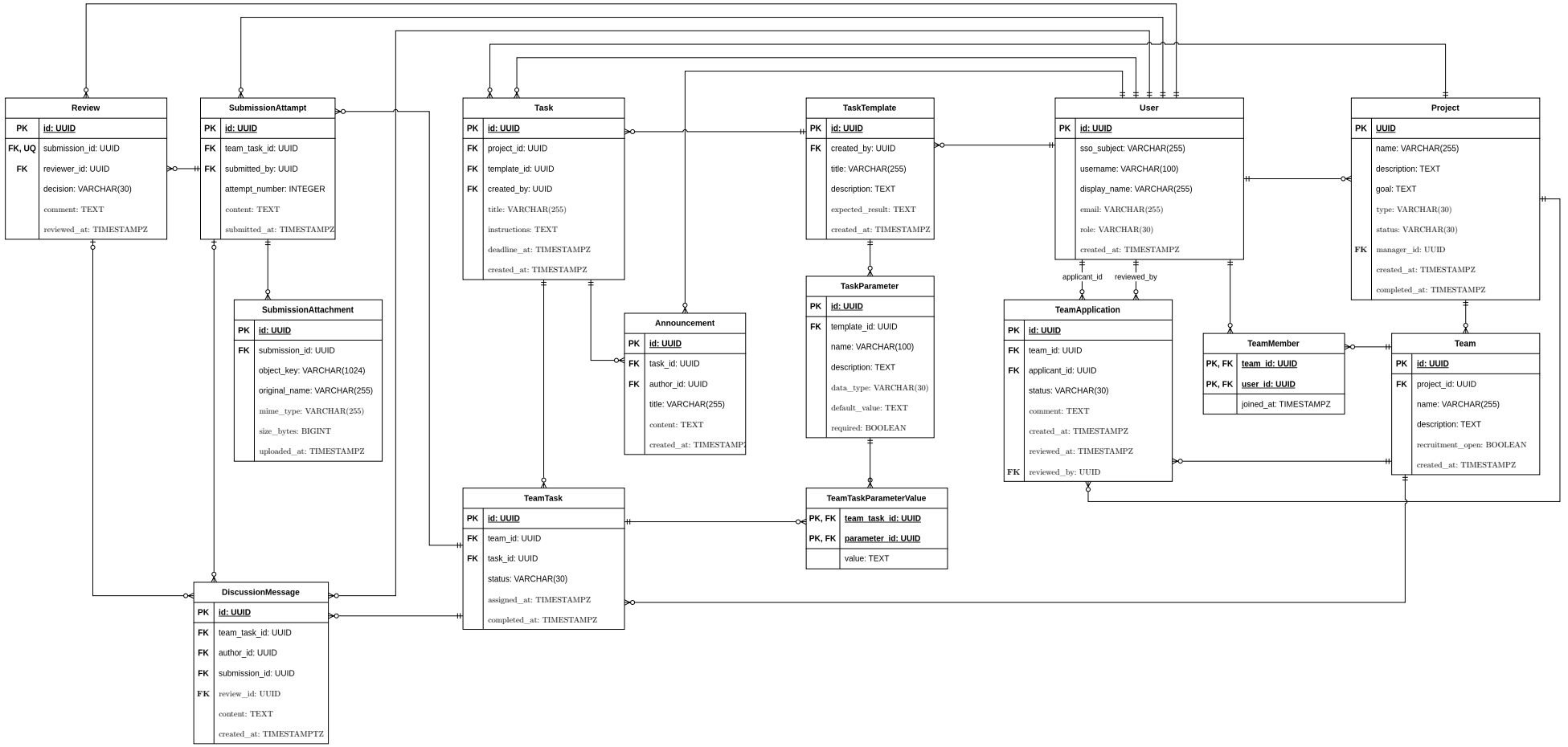


Рис. 1. Даталогическая модель сущностей

2. Таблицы

Физическая модель данных реализована в PostgreSQL.

Для `id-PK` используется `uuid-v7` для индексации по времени создания.

Скрипт `01_schema.sql` – создает таблицы, внешние ключи, ограничения уникальности и прочие проверки целостности.

2.1. Пользователи и проекты

2.1.1. User

Учетная запись пользователя платформы.

2.1.1.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`
sso_subject	`VARCHAR(255)`	`NOT NULL UNIQUE`
username	`VARCHAR(100)`	`NOT NULL UNIQUE`
display_name	`VARCHAR(255)`	`NOT NULL`
email	`VARCHAR(255)`	`NOT NULL UNIQUE`
role	`VARCHAR(30)`	`NOT NULL CHECK (role IN ('student', 'teacher', 'admin'))`
created_at	`TIMESTAMPTZ`	`NOT NULL DEFAULT now()`

2.1.1.2. DDL

```sql
CREATE TABLE app_user (
    id           UUID PRIMARY KEY DEFAULT uuidv7(),
    sso_subject  VARCHAR(255) NOT NULL UNIQUE,
    username     VARCHAR(100) NOT NULL UNIQUE,
    display_name VARCHAR(255) NOT NULL,
    email        VARCHAR(255) NOT NULL UNIQUE,
    role         VARCHAR(30) NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_user_role
      CHECK (role IN ('student', 'teacher', 'admin'))
);
```

2.1.1.3. Индексы

Поле	Индекс	Обоснование
`id`	`B-tree`	Поиск пользователя по идентификатору. Создаётся автоматически для первичного ключа.
`sso_subject`, `username`, `email`	`B-tree`	Проверка уникальности и поиск пользователя при авторизации.

2.1.2. Project

Учебный проект, объединяющий команды и задания под управлением пользователя.

2.1.2.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`
name	`VARCHAR(255)`	`NOT NULL`
description	`TEXT`	`NULL`
goal	`TEXT`	`NULL`
type	`VARCHAR(30)`	`NOT NULL CHECK (type IN ('educational', 'development'))`
status	`VARCHAR(30)`	`NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'active', 'completed', 'archived'))`
manager_id (FK)	`UUID`	`NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT`
created_at	`TIMESTAMPTZ`	`NOT NULL DEFAULT now()`
completed_at	`TIMESTAMPTZ`	`NULL CHECK (completed_at IS NULL OR completed_at >= created_at)`

2.1.2.2. DDL

```
CREATE TABLE project (
    id           UUID PRIMARY KEY DEFAULT uuidv7(),
    name         VARCHAR(255) NOT NULL,
    description  TEXT,
    goal         TEXT,
```

```
    type         VARCHAR(30) NOT NULL,
    status       VARCHAR(30) NOT NULL DEFAULT 'draft',
    manager_id   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ,

    CONSTRAINT chk_project_type
        CHECK (type IN ('educational', 'development')),

    CONSTRAINT chk_project_status
        CHECK (status IN ('draft', 'active', 'completed', 'archived')),

    CONSTRAINT chk_project_completion
        CHECK (completed_at IS NULL OR completed_at >= created_at)
);
```

2.1.2.3. Индексы

Поле	Индекс	Обоснование
manager_id	B-tree	Получение проектов управляющего.

2.1.3. Team

Команда пользователей, созданная внутри проекта.

2.1.3.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
project_id (FK)	UUID	NOT NULL REFERENCES Project(id) ON DELETE CASCADE
name	VARCHAR(255)	NOT NULL UNIQUE (project_id, name)
description	TEXT	NULL
recruitment_open	BOOLEAN	NOT NULL DEFAULT true
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.1.3.2. DDL

```
CREATE TABLE team (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    project_id      UUID NOT NULL
```

```
        REFERENCES project(id) ON DELETE CASCADE,
    name            VARCHAR(255) NOT NULL,
    description     TEXT,
    recruitment_open BOOLEAN NOT NULL DEFAULT true,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT uq_team_name
        UNIQUE (project_id, name)
);
```

2.1.3.3. Индексы

Поле	Индекс	Обоснование
project_id, name	B-tree	Получение команд проекта и проверка уникальности их названий.
project_id	B-tree (частичный)	Поиск команд с открытым набором.

2.1.4. TeamMember

Связь пользователя с командой, фиксирующая участие и дату вступления.

2.1.4.1. Поля и ограничения

Поле	Тип	Ограничения
team_id (PK, FK)	UUID	PRIMARY KEY (team_id, user_id) REFERENCES Team(id) ON DELETE CASCADE
user_id (PK, FK)	UUID	REFERENCES "User"(id) ON DELETE CASCADE
joined_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.1.4.2. DDL

```
CREATE TABLE team_member (
    team_id   UUID NOT NULL
        REFERENCES team(id) ON DELETE CASCADE,
    user_id   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE CASCADE,
    joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    PRIMARY KEY (team_id, user_id)
);
```

2.1.4.3. Индексы

Поле	Индекс	Обоснование

team_id, user_id	B-tree	Получение участников команды и исключение повторного членства.
user_id	B-tree	Получение команд пользователя.

2.1.5. TeamApplication

Заявка пользователя на вступление в выбранную команду.

2.1.5.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
team_id (FK)	UUID	NOT NULL REFERENCES Team(id) ON DELETE CASCADE
applicant_id (FK)	UUID	NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT
status	VARCHAR(30)	NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'accepted', 'rejected'))
comment	TEXT	NULL
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()
reviewed_at	TIMESTAMPTZ	NULL CHECK (reviewed_at IS NULL OR reviewed_at >= created_at)
reviewed_by (FK)	UUID	NULL REFERENCES "User"(id) ON DELETE RESTRICT

2.1.5.2. DDL

```
CREATE TABLE team_application (
    id           UUID PRIMARY KEY DEFAULT uuidv7(),
    team_id      UUID NOT NULL
        REFERENCES team(id) ON DELETE CASCADE,
    applicant_id UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    status       VARCHAR(30) NOT NULL DEFAULT 'pending',
    comment      TEXT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    reviewed_at  TIMESTAMPTZ,
    reviewed_by  UUID
        REFERENCES app_user(id) ON DELETE RESTRICT,
```

```
    CONSTRAINT chk_application_status
        CHECK (status IN ('pending', 'accepted', 'rejected')),

    CONSTRAINT chk_application_review
        CHECK (
            (
                status = 'pending'
                AND reviewed_at IS NULL
                AND reviewed_by IS NULL
            )
            OR
            (
                status IN ('accepted', 'rejected')
                AND reviewed_at IS NOT NULL
                AND reviewed_by IS NOT NULL
            )
        )
);
```

2.1.5.3. Индексы

Поле	Индекс	Обоснование
`team_id`, `status`, `created_at`	`B-tree`	Фильтрация и сортировка заявок команды по состоянию.
`team_id`, `applicant_id`	`B-tree (частичный)`	Частичный индекс запрещает несколько активных заявок одного пользователя в команду.

2.2. Задания и назначения

2.2.1. TaskTemplate

Переиспользуемый шаблон, определяющий содержание и ожидаемый результат задания.

2.2.1.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`

created_by (FK)	UUID	NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT
title	VARCHAR(255)	NOT NULL CHECK (btrim(title) <> '')
description	TEXT	NULL
expected_result	TEXT	NULL
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.2.1.2. DDL

```
CREATE TABLE task_template (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    created_by      UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    title           VARCHAR(255) NOT NULL,
    description     TEXT,
    expected_result TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_template_title
        CHECK (btrim(title) <> '')
);
```

2.2.1.3. Индексы

Поле	Индекс	Обоснование
created_by	B-tree	Получение шаблонов пользователя.

2.2.2. TaskParameter

Настраиваемый параметр шаблона задания.

2.2.2.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
template_id (FK)	UUID	NOT NULL REFERENCES TaskTemplate(id) ON DELETE CASCADE
name	VARCHAR(100)	NOT NULL UNIQUE (template_id, name)
description	TEXT	NULL

data_type	VARCHAR(30)	NOT NULL CHECK (data_type IN ('text', 'integer', 'decimal', 'boolean', 'date'))
default_value	TEXT	NULL TRIGGER validate_default_value_type
required	BOOLEAN	NOT NULL DEFAULT false

2.2.2.2. DDL

```
CREATE TABLE task_parameter (
    id            UUID PRIMARY KEY DEFAULT uuidv7(),
    template_id   UUID NOT NULL
        REFERENCES task_template(id) ON DELETE CASCADE,
    name          VARCHAR(100) NOT NULL,
    description   TEXT,
    data_type     VARCHAR(30) NOT NULL,
    default_value TEXT,
    required      BOOLEAN NOT NULL DEFAULT false,

    CONSTRAINT uq_template_parameter
        UNIQUE (template_id, name),

    CONSTRAINT chk_parameter_type
        CHECK (
            data_type IN (
                'text',
                'integer',
                'decimal',
                'boolean',
                'date'
            )
        )
);
```

2.2.2.3. Индексы

Поле	Индекс	Обоснование
template_id, name	B-tree	Получение параметров шаблона и проверка уникальности имени параметра.

2.2.3. Task

Задание проекта, созданное на основе выбранного шаблона.

2.2.3.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
project_id (FK)	UUID	NOT NULL REFERENCES Project(id) ON DELETE CASCADE
template_id (FK)	UUID	NOT NULL REFERENCES TaskTemplate(id) ON DELETE RESTRICT
created_by (FK)	UUID	NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT
title	VARCHAR(255)	NOT NULL CHECK (btrim(title) <> '')
instructions	TEXT	NOT NULL
deadline_at	TIMESTAMPTZ	NOT NULL CHECK (deadline_at > created_at)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.2.3.2. DDL

```
CREATE TABLE task (
    id           UUID PRIMARY KEY DEFAULT uuidv7(),
    project_id   UUID NOT NULL
        REFERENCES project(id) ON DELETE CASCADE,
    template_id  UUID NOT NULL
        REFERENCES task_template(id) ON DELETE RESTRICT,
    created_by   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    title        VARCHAR(255) NOT NULL,
    instructions TEXT NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    deadline_at  TIMESTAMPTZ NOT NULL,

    CONSTRAINT chk_task_title
        CHECK (btrim(title) <> ''),

    CONSTRAINT chk_task_deadline
        CHECK (deadline_at > created_at)
);
```

2.2.3.3. Индексы

Поле	Индекс	Обоснование
`project_id, deadline_at`	`B-tree`	Получение заданий проекта по сроку выполнения.

2.2.4. TeamTask

Назначение задания конкретной команде с собственным состоянием выполнения.

2.2.4.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`
team_id (FK)	`UUID`	`NOT NULL REFERENCES Team(id) ON DELETE CASCADE`
task_id (FK)	`UUID`	`NOT NULL REFERENCES Task(id) ON DELETE CASCADE UNIQUE (team_id, task_id) TRIGGER validate_team_task_project`
status	`VARCHAR(30)`	`NOT NULL DEFAULT 'assigned' CHECK (status IN ('assigned', 'in_progress', 'submitted', 'revision_required', 'completed'))`
assigned_at	`TIMESTAMPTZ`	`NOT NULL DEFAULT now()`
completed_at	`TIMESTAMPTZ`	`NULL CHECK (completed_at IS NULL OR completed_at >= assigned_at)`

2.2.4.2. DDL

```
CREATE TABLE team_task (
    id           UUID PRIMARY KEY DEFAULT uuidv7(),
    team_id      UUID NOT NULL
        REFERENCES team(id) ON DELETE CASCADE,
    task_id      UUID NOT NULL
        REFERENCES task(id) ON DELETE CASCADE,
    status       VARCHAR(30) NOT NULL DEFAULT 'assigned',
    assigned_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ,

    CONSTRAINT uq_team_task
```

```
        UNIQUE (team_id, task_id),

    CONSTRAINT chk_team_task_status
        CHECK (
            status IN (
                'assigned',
                'in_progress',
                'submitted',
                'revision_required',
                'completed'
            )
        ),

    CONSTRAINT chk_team_task_completion
        CHECK (completed_at IS NULL OR completed_at >= assigned_at)
);
```

2.2.4.3. Индексы

Поле	Индекс	Обоснование
`team_id, task_id`	`B-tree`	Получение назначений команды и исключение повторного назначения задания.
`task_id`	`B-tree`	Получение команд, которым назначено задание.

2.2.5. TeamTaskParameterValue

Значение параметра задания, настроенное для конкретной команды.

2.2.5.1. Поля и ограничения

Поле	Тип	Ограничения
team_task_id (PK, FK)	`UUID`	`PRIMARY KEY (team_task_id, parameter_id)` `REFERENCES TeamTask(id) ON DELETE CASCADE`
parameter_id (PK, FK)	`UUID`	`REFERENCES TaskParameter(id) ON DELETE CASCADE` `TRIGGER validate_team_task_parameter`
value	`TEXT`	`NOT NULL` `TRIGGER validate_parameter_value`

2.2.5.2. DDL

```
CREATE TABLE team_task_parameter_value (
    team_task_id UUID NOT NULL
        REFERENCES team_task(id) ON DELETE CASCADE,
```

```
    parameter_id UUID NOT NULL
        REFERENCES task_parameter(id) ON DELETE CASCADE,
    value        TEXT NOT NULL,

    PRIMARY KEY (team_task_id, parameter_id)
);
```

2.2.5.3. Индексы

Поле	Индекс	Обоснование
`team_task_id, parameter_id`	`B-tree`	Получение значений параметров назначения и исключение повторных значений.

2.3. Сдача и проверка

2.3.1. SubmissionAttempt

Отдельная попытка сдачи назначенного команде задания.

2.3.1.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`
team_task_id (FK)	`UUID`	`NOT NULL REFERENCES TeamTask(id) ON DELETE CASCADE`
submitted_by (FK)	`UUID`	`NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT` `TRIGGER validate_submission_author`
attempt_number	`INTEGER`	`NOT NULL CHECK (attempt_number > 0) UNIQUE (team_task_id, attempt_number)`
content	`TEXT`	`NULL`
submitted_at	`TIMESTAMPTZ`	`NOT NULL DEFAULT now()`

2.3.1.2. DDL

```
CREATE TABLE submission_attempt (
    id             UUID PRIMARY KEY DEFAULT uuidv7(),
    team_task_id   UUID NOT NULL
        REFERENCES team_task(id) ON DELETE CASCADE,
```

```
    submitted_by   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    attempt_number INTEGER NOT NULL,
    content        TEXT,
    submitted_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT uq_submission_attempt
        UNIQUE (team_task_id, attempt_number),

    CONSTRAINT chk_attempt_number
        CHECK (attempt_number > 0)
);
```

2.3.1.3. Индексы

Поле	Индекс	Обоснование
`team_task_id, attempt_number`	`B-tree`	Получение истории попыток назначения в порядке их номеров.

2.3.2. SubmissionAttachment

Метаданные файла попытки сдачи, хранящегося в объектном хранилище.

2.3.2.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY` `DEFAULT uuidv7()`

submission_id (FK)	`UUID`	`NOT NULL`
		`REFERENCES SubmissionAttempt(id)`
		`ON DELETE CASCADE`

object_key	`VARCHAR(1024)`	`NOT NULL`
		`UNIQUE`
		`CHECK (btrim(object_key) <> '')`

original_name	`VARCHAR(255)`	`NOT NULL` `CHECK (btrim(original_name) <> '')`
mime_type	`VARCHAR(255)`	`NOT NULL`
size_bytes	`BIGINT`	`NOT NULL` `CHECK (size_bytes > 0)`
uploaded_at	`TIMESTAMPTZ`	`NOT NULL` `DEFAULT now()`

2.3.2.2. DDL

```
CREATE TABLE submission_attachment (
    id            UUID PRIMARY KEY DEFAULT uuidv7(),
    submission_id UUID NOT NULL
```

```
        REFERENCES submission_attempt(id) ON DELETE CASCADE,
    object_key    VARCHAR(1024) NOT NULL UNIQUE,
    original_name VARCHAR(255) NOT NULL,
    mime_type     VARCHAR(255) NOT NULL,
    size_bytes    BIGINT NOT NULL,
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_attachment_key
        CHECK (btrim(object_key) <> ''),

    CONSTRAINT chk_attachment_name
        CHECK (btrim(original_name) <> ''),

    CONSTRAINT chk_attachment_size
        CHECK (size_bytes > 0)
);
```

2.3.2.3. Индексы

Поле	Индекс	Обоснование
`object_key`	`B-tree`	Поиск метаданных по уникальному ключу объекта в MinIO.
`submission_id`	`B-tree`	Получение файлов попытки сдачи.

2.3.3. Review

Результат проверки конкретной попытки сдачи.

2.3.3.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	`UUID`	`PRIMARY KEY DEFAULT uuidv7()`
submission_id (FK, UQ)	`UUID`	`NOT NULL UNIQUE REFERENCES SubmissionAttempt(id) ON DELETE CASCADE`
reviewer_id (FK)	`UUID`	`NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT TRIGGER validate_reviewer`
decision	`VARCHAR(30)`	`NOT NULL CHECK (decision IN ('accepted', 'revision_required'))`

		TRIGGER update_team_task_status
comment	TEXT	NULL CHECK (decision = 'accepted' OR nullif(btrim(comment), '') IS NOT NULL)
reviewed_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.3.3.2. DDL

```
CREATE TABLE review (
    id            UUID PRIMARY KEY DEFAULT uuidv7(),
    submission_id UUID NOT NULL UNIQUE
        REFERENCES submission_attempt(id) ON DELETE CASCADE,
    reviewer_id   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    decision      VARCHAR(30) NOT NULL,
    comment       TEXT,
    reviewed_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_review_decision
        CHECK (decision IN ('accepted', 'revision_required')),

    CONSTRAINT chk_revision_comment
        CHECK (
            decision = 'accepted'
            OR nullif(btrim(comment), '') IS NOT NULL
        )
);
```

2.3.3.3. Индексы

Поле	Индекс	Обоснование
submission_id	B-tree	Быстрое получение результата проверки по попытке и ограничение одной проверки на попытку.

2.4. Объявления и обсуждение

2.4.1. Announcement

Объявление, опубликованное для всех команд, выполняющих задание.

2.4.1.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
task_id (FK)	UUID	NOT NULL REFERENCES Task(id) ON DELETE CASCADE
author_id (FK)	UUID	NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT
title	VARCHAR(255)	NOT NULL CHECK (btrim(title) <> '')
content	TEXT	NOT NULL CHECK (btrim(content) <> '')
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.4.1.2. DDL

```
CREATE TABLE announcement (
    id          UUID PRIMARY KEY DEFAULT uuidv7(),
    task_id     UUID NOT NULL
        REFERENCES task(id) ON DELETE CASCADE,
    author_id   UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    title       VARCHAR(255) NOT NULL,
    content     TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_announcement_title
        CHECK (btrim(title) <> ''),

    CONSTRAINT chk_announcement_content
        CHECK (btrim(content) <> '')
);
```

2.4.1.3. Индексы

Поле	Индекс	Обоснование
task_id, created_at	B-tree	Вывод объявлений задания от новых к старым.

2.4.2. DiscussionMessage

Сообщение в обсуждении назначенного задания, сдачи или результата проверки.

2.4.2.1. Поля и ограничения

Поле	Тип	Ограничения
id (PK)	UUID	PRIMARY KEY DEFAULT uuidv7()
team_task_id (FK)	UUID	NOT NULL REFERENCES TeamTask(id) ON DELETE CASCADE TRIGGER validate_message_scope
author_id (FK)	UUID	NOT NULL REFERENCES "User"(id) ON DELETE RESTRICT
submission_id (FK)	UUID	NULL REFERENCES SubmissionAttempt(id) ON DELETE CASCADE
review_id (FK)	UUID	NULL REFERENCES Review(id) ON DELETE CASCADE CHECK (num_nonnulls(submission_id, review_id) <= 1)
content	TEXT	NOT NULL CHECK (btrim(content) <> '')
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

2.4.2.2. DDL

```
CREATE TABLE discussion_message (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    team_task_id    UUID NOT NULL
        REFERENCES team_task(id) ON DELETE CASCADE,
    author_id       UUID NOT NULL
        REFERENCES app_user(id) ON DELETE RESTRICT,
    submission_id UUID
        REFERENCES submission_attempt(id) ON DELETE SET NULL,
    review_id       UUID
        REFERENCES review(id) ON DELETE SET NULL,
    content         TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT chk_message_target
        CHECK (num_nonnulls(submission_id, review_id) <= 1),

    CONSTRAINT chk_message_content
```

```
        CHECK (btrim(content) <> '')
);
```

2.4.2.3. Индексы

Поле	Индекс	Обоснование
`team_task_id`, `created_at`	`B-tree`	Последовательная загрузка обсуждения назначения.

3. Триггеры

3.1. `trg_validate_default_value_type`

3.1.1. Зачем

Проверяет typecast и не дает сохранить неверный тип.

3.1.2. Код

```
CREATE OR REPLACE FUNCTION is_parameter_value_valid(
    checked_type VARCHAR,
    checked_value TEXT
)
RETURNS BOOLEAN
LANGUAGE plpgsql
IMMUTABLE
AS $$
BEGIN
    IF checked_value IS NULL OR checked_type = 'text' THEN
        RETURN TRUE;
    END IF;

    CASE checked_type
        WHEN 'integer' THEN PERFORM checked_value::BIGINT;
        WHEN 'decimal' THEN PERFORM checked_value::NUMERIC;
        WHEN 'boolean' THEN PERFORM checked_value::BOOLEAN;
        WHEN 'date' THEN PERFORM checked_value::DATE;
        ELSE RETURN FALSE;
    END CASE;

    RETURN TRUE;
EXCEPTION
    WHEN invalid_text_representation
      OR numeric_value_out_of_range
      OR datetime_field_overflow THEN
        RETURN FALSE;
END;
$$;

CREATE OR REPLACE FUNCTION validate_default_value_type()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT is_parameter_value_valid(
```

```
        NEW.data_type,
        NEW.default_value
    ) THEN
        RAISE EXCEPTION
            'Default value does not match parameter type';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_default_value_type
BEFORE INSERT OR UPDATE OF data_type, default_value
ON task_parameter
FOR EACH ROW
EXECUTE FUNCTION validate_default_value_type();
```

3.2. trg_validate_team_task_project

3.2.1. Зачем

Проверяет, что команда и задание относятся к одному проекту.

3.2.2. Код

```
CREATE OR REPLACE FUNCTION validate_team_task_project()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    team_project_id UUID;
    task_project_id UUID;
BEGIN
    SELECT project_id INTO team_project_id
    FROM team
    WHERE id = NEW.team_id;

    SELECT project_id INTO task_project_id
    FROM task
    WHERE id = NEW.task_id;

    IF team_project_id IS DISTINCT FROM task_project_id THEN
        RAISE EXCEPTION
            'Task and team must belong to the same project';
    END IF;

    RETURN NEW;
```

```
END;
$$;

CREATE TRIGGER trg_validate_team_task_project
BEFORE INSERT OR UPDATE OF team_id, task_id
ON team_task
FOR EACH ROW
EXECUTE FUNCTION validate_team_task_project();
```

3.3. trg_validate_team_task_parameter

3.3.1. Зачем

Не дает использовать параметр из другого шаблона.

3.3.2. Код

```
CREATE OR REPLACE FUNCTION validate_team_task_parameter()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM team_task tt
        JOIN task t ON t.id = tt.task_id
        JOIN task_parameter tp
            ON tp.id = NEW.parameter_id
           AND tp.template_id = t.template_id
        WHERE tt.id = NEW.team_task_id
    ) THEN
        RAISE EXCEPTION
            'Parameter does not belong to the task template';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_team_task_parameter
BEFORE INSERT OR UPDATE OF team_task_id, parameter_id
ON team_task_parameter_value
FOR EACH ROW
EXECUTE FUNCTION validate_team_task_parameter();
```

3.4. trg_validate_parameter_value

3.4.1. Зачем

Проверяет, что значение подходит под тип параметра.

3.4.2. Код

```
CREATE OR REPLACE FUNCTION validate_parameter_value()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    parameter_type VARCHAR(30);
BEGIN
    SELECT data_type INTO parameter_type
    FROM task_parameter
    WHERE id = NEW.parameter_id;

    IF NOT is_parameter_value_valid(
        parameter_type,
        NEW.value
    ) THEN
        RAISE EXCEPTION
            'Parameter value does not match declared type';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_parameter_value
BEFORE INSERT OR UPDATE OF parameter_id, value
ON team_task_parameter_value
FOR EACH ROW
EXECUTE FUNCTION validate_parameter_value();
```

3.5. trg_validate_submission_author

3.5.1. Зачем

Не дает сдать работу за чужую команду или после завершения задания.

3.5.2. Код

```
CREATE OR REPLACE FUNCTION validate_submission_author()
RETURNS TRIGGER
LANGUAGE plpgsql
```

```
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM team_task tt
        JOIN team_member tm ON tm.team_id = tt.team_id
        WHERE tt.id = NEW.team_task_id
          AND tm.user_id = NEW.submitted_by
          AND tt.status <> 'completed'
    ) THEN
        RAISE EXCEPTION
            'Submission author must be a team member and task must be
open';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_submission_author
BEFORE INSERT OR UPDATE OF team_task_id, submitted_by
ON submission_attempt
FOR EACH ROW
EXECUTE FUNCTION validate_submission_author();
```

3.6. trg_mark_team_task_submitted

3.6.1. Зачем

После сдачи автоматически меняет состояние назначения на `submitted`.

3.6.2. Код

```
CREATE OR REPLACE FUNCTION mark_team_task_submitted()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE team_task
    SET status = 'submitted',
        completed_at = NULL
    WHERE id = NEW.team_task_id;

    RETURN NEW;
END;
$$;
```

```
CREATE TRIGGER trg_mark_team_task_submitted
AFTER INSERT
ON submission_attempt
FOR EACH ROW
EXECUTE FUNCTION mark_team_task_submitted();
```

3.7. trg_validate_reviewer

3.7.1. Зачем

Не дает проверить работу пользователю, который не управляет проектом.

3.7.2. Код

```
CREATE OR REPLACE FUNCTION validate_reviewer()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM submission_attempt sa
        JOIN team_task tt ON tt.id = sa.team_task_id
        JOIN task t ON t.id = tt.task_id
        JOIN project p ON p.id = t.project_id
        WHERE sa.id = NEW.submission_id
          AND p.manager_id = NEW.reviewer_id
    ) THEN
        RAISE EXCEPTION
            'Only the project manager may review a submission';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_reviewer
BEFORE INSERT OR UPDATE OF submission_id, reviewer_id
ON review
FOR EACH ROW
EXECUTE FUNCTION validate_reviewer();
```

3.8. trg_update_team_task_status

3.8.1. Зачем

После проверки автоматически ставит `completed` или `revision_required`.

3.8.2. Код

```
CREATE OR REPLACE FUNCTION update_team_task_status()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE team_task tt
    SET status = CASE
            WHEN NEW.decision = 'accepted' THEN 'completed'
            ELSE 'revision_required'
        END,
        completed_at = CASE
            WHEN NEW.decision = 'accepted' THEN NEW.reviewed_at
            ELSE NULL
        END
    FROM submission_attempt sa
    WHERE sa.id = NEW.submission_id
      AND tt.id = sa.team_task_id;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_update_team_task_status
AFTER INSERT OR UPDATE OF decision
ON review
FOR EACH ROW
EXECUTE FUNCTION update_team_task_status();
```

3.9. trg_validate_message_scope

3.9.1. Зачем

Проверяет, что сообщение, сдача и проверка относятся к одному назначению.

3.9.2. Код

```
CREATE OR REPLACE FUNCTION validate_message_scope()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.submission_id IS NOT NULL AND NOT EXISTS (
        SELECT 1
        FROM submission_attempt sa
```

```
        WHERE sa.id = NEW.submission_id
          AND sa.team_task_id = NEW.team_task_id
    ) THEN
        RAISE EXCEPTION
            'Submission does not belong to the selected team task';
    END IF;

    IF NEW.review_id IS NOT NULL AND NOT EXISTS (
        SELECT 1
        FROM review r
        JOIN submission_attempt sa ON sa.id = r.submission_id
        WHERE r.id = NEW.review_id
          AND sa.team_task_id = NEW.team_task_id
    ) THEN
        RAISE EXCEPTION
            'Review does not belong to the selected team task';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_message_scope
BEFORE INSERT OR UPDATE OF team_task_id, submission_id, review_id
ON discussion_message
FOR EACH ROW
EXECUTE FUNCTION validate_message_scope();
```

4. Функции и процедуры

Для «упрощения» жизни можно создать набор из атомарных операций над бд.

4.1. `apply_to_team`

4.1.1. Зачем

Создает заявку, проверяет, что набор открыт, а пользователь еще не состоит в команде.

4.1.2. Код

```
CREATE OR REPLACE FUNCTION apply_to_team(
    p_team_id UUID,
    p_applicant_id UUID,
    p_comment TEXT DEFAULT NULL
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    application_id UUID;
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM team
        WHERE id = p_team_id
          AND recruitment_open = true
    ) THEN
        RAISE EXCEPTION
            'Team does not exist or recruitment is closed';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM team_member
        WHERE team_id = p_team_id
          AND user_id = p_applicant_id
    ) THEN
        RAISE EXCEPTION
            'User is already a member of this team';
    END IF;

    INSERT INTO team_application (
        team_id,
```

```
        applicant_id,
        comment
    )
    VALUES (
        p_team_id,
        p_applicant_id,
        p_comment
    )
    RETURNING id INTO application_id;

    RETURN application_id;
EXCEPTION WHEN unique_violation THEN RAISE EXCEPTION 'User already has
a pending application for this team';
END;
$$;
```

4.2. review_team_application

4.2.1. Зачем

Принимает или отклоняет заявку. При принятии добавляет пользователя в команду.

4.2.2. Код

```
CREATE OR REPLACE PROCEDURE review_team_application(
    p_application_id UUID,
    p_reviewer_id UUID,
    p_decision VARCHAR(30),
    p_comment TEXT DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    application_team_id UUID;
    application_user_id UUID;
    application_status VARCHAR(30);
    project_manager_id UUID;
BEGIN
    IF p_decision NOT IN ('accepted', 'rejected') THEN
        RAISE EXCEPTION
            'Decision must be accepted or rejected';
    END IF;

    SELECT
        ta.team_id,
        ta.applicant_id,
```

```
        ta.status,
        p.manager_id
    INTO
        application_team_id,
        application_user_id,
        application_status,
        project_manager_id
    FROM team_application ta
    JOIN team t ON t.id = ta.team_id
    JOIN project p ON p.id = t.project_id
    WHERE ta.id = p_application_id
    FOR UPDATE OF ta;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Application not found';
    END IF;

    IF application_status <> 'pending' THEN
        RAISE EXCEPTION 'Application is already reviewed';
    END IF;

    IF project_manager_id <> p_reviewer_id THEN
        RAISE EXCEPTION
            'Only the project manager may review this application';
    END IF;

    UPDATE team_application
    SET status = p_decision,
        comment = COALESCE(p_comment, comment),
        reviewed_at = now(),
        reviewed_by = p_reviewer_id
    WHERE id = p_application_id;

    IF p_decision = 'accepted' THEN
        INSERT INTO team_member (team_id, user_id)
        VALUES (application_team_id, application_user_id)
        ON CONFLICT DO NOTHING;
    END IF;
END;
$$;
```

4.3. submit_task

4.3.1. Зачем

Создает новую попытку сдачи и сама определяет номер попытки (намеренная денормализация)

4.3.2. Код

```
CREATE OR REPLACE FUNCTION submit_task(
    p_team_task_id UUID,
    p_submitted_by UUID,
    p_content TEXT DEFAULT NULL
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    next_attempt INTEGER;
    submission_id UUID;
BEGIN
    PERFORM 1
    FROM team_task
    WHERE id = p_team_task_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Team task not found';
    END IF;

    SELECT COALESCE(MAX(attempt_number), 0) + 1
    INTO next_attempt
    FROM submission_attempt
    WHERE team_task_id = p_team_task_id;

    INSERT INTO submission_attempt (
        team_task_id,
        submitted_by,
        attempt_number,
        content
    )
    VALUES (
        p_team_task_id,
        p_submitted_by,
        next_attempt,
        p_content
    )
```

```
    RETURNING id INTO submission_id;

    RETURN submission_id;
END;
$$;
```

4.4. review_submission

4.4.1. Зачем

Создает результат проверки и не дает проверить одну сдачу дважды.

4.4.2. Код

```
CREATE OR REPLACE FUNCTION review_submission(
    p_submission_id UUID,
    p_reviewer_id UUID,
    p_decision VARCHAR(30),
    p_comment TEXT DEFAULT NULL
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    review_id UUID;
BEGIN
    IF p_decision NOT IN ('accepted', 'revision_required') THEN
        RAISE EXCEPTION
            'Decision must be accepted or revision_required';
    END IF;

    PERFORM 1
    FROM submission_attempt
    WHERE id = p_submission_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Submission not found';
    END IF;

    INSERT INTO review (
        submission_id,
        reviewer_id,
        decision,
        comment
    )
    VALUES (
```

```
        p_submission_id,
        p_reviewer_id,
        p_decision,
        p_comment
    )
    RETURNING id INTO review_id;

    RETURN review_id;
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION
            'Submission has already been reviewed';
END;
$$;
```

5. Заполнение тестовыми данными

Скрипт для заполнения тестовыми данными полностью сгенерен LLM. Промпт:

```
Сгенерируй sql (postgres) скрипт для заполнения всех таблиц тестовыми данными. Используй корректные определения сущностей. Вот описание сущностей <pasted typst description>
```

Исходный код можно посмотреть в репозитории в директории `sql/` (ветка `feat/stage2`): https://github.com/xtern0o/is-course